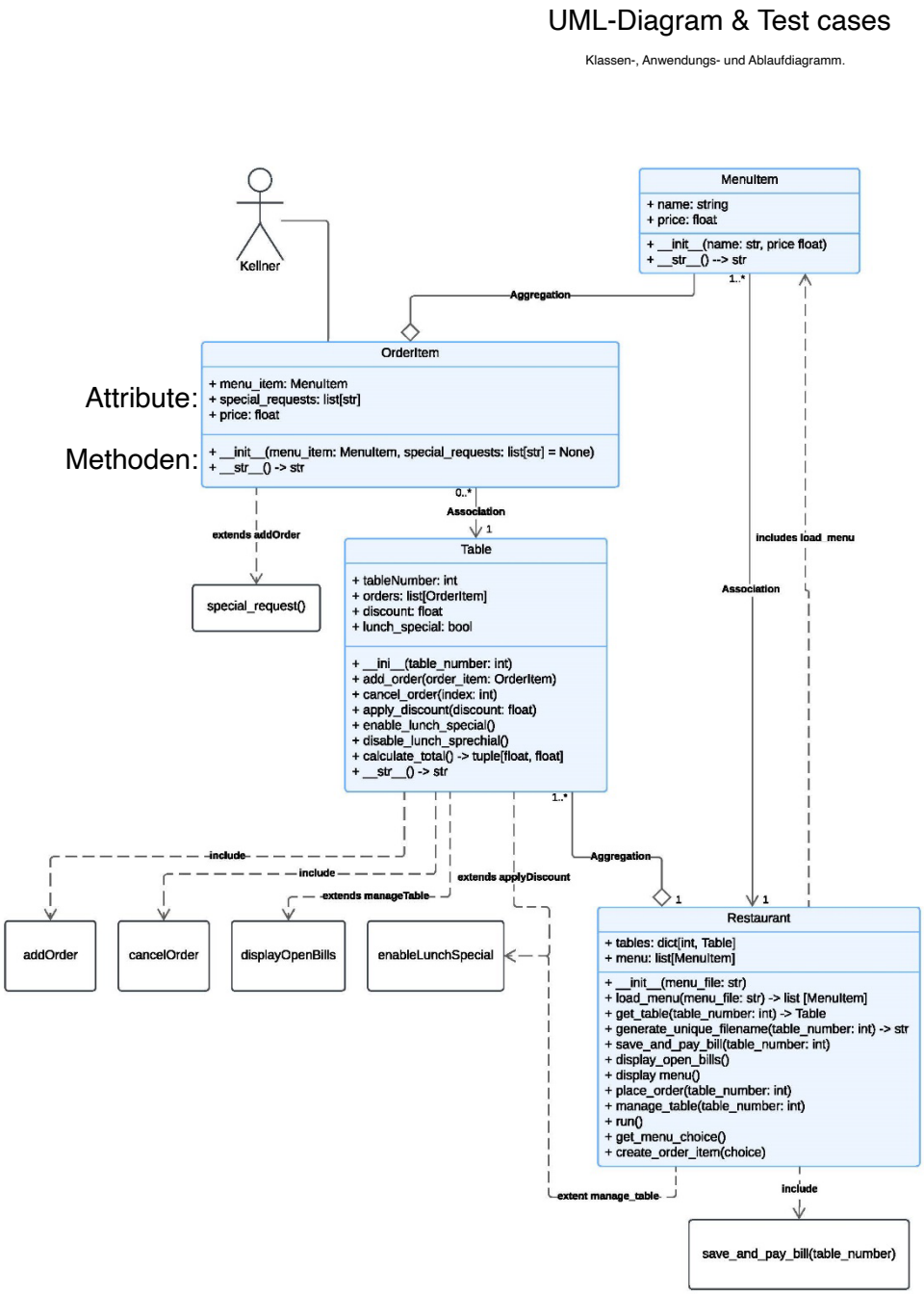
Dokumentation Omikron – Uebung 7 Teamarbeit

UML-Klassendiagramm



Dokumentation – Coding

Die Klasse MenuItem (class MenuItem) dient dazu den Namen und den Preis eines Gerichts/Getränks zu speichern und diesen anzuzeigen, unter der Klasse steht die Methode def __init__, welches dem Zweck dient genau diese Daten zu speichern und die Methode def __str__ gibt dieses Menüelement als Text zurück (Textform: formatierte Zeichenkette, die den Namen

des Gerichts (self.name) und den Preis (self.price) mit zwei Nachkommastellen und dem Euro-Zeichen (€) anzeigt).

Die Klasse OrderItem (class OrderItem) repräsentiert ein einzelnes bestelltes Menüelement für einen Tisch. Sie kombiniert ein Menüelement (vom Typ MenuItem) mit einer Liste von Sonderwünschen (special requests).

Die Methode __init__ initialisiert eine neue Tischnummer (self.table_number), eine Liste in der alle Bestellungen gespeichert werden (self.orders),), eine Liste zur Speicherung aller Bestellungen (self.orders), einen Rabattwert in Prozent (self.discount) und einen booleschen Wert, der angibt, ob das Lunch-Special aktiviert ist (standardmäßig immer deaktiviert bis zur Aktivierung), falls dieser Lunch-Special aktiviert ist, kommt es zu zusätzlichen Kosten von 1€ (**1.0 for req in self.special_requests if "extra" in req.lower())**

Mithilfe der add_order Methode können Bestellungen (ein OrderItem-Objekt) zur Bestellliste (self.orders) des Tisches hinzugefügt werden.

Die Methode cancel_order entfernt hingegen zum add_order eine Bestellung aus der Bestellliste eines Tisches, basierend auf deren Position in der Liste. **0 <= index < len(self.orders)** überprüft, ob der angegebene index im gültigen Bereich der Bestellliste liegt, während **removed_item = self.orders.pop(index)**, falls der Index gültig ist, eine Bestellung an dieser Position aus der Liste entfernt und in der Variablen removed_item gespeichert werden. Entweder wird eine Nachricht für Erfolg oder für Misserfolg wird ausgegeben.

Die Methode apply_discount erlaubt es, uns einen Prozentsatz-Rabatt auf den Gesamtbetrag des Tisches anzuwenden. Dabei gibt der Discount einen float-Wert wieder, der den Rabatt in Prozent darstellt.

Die Methode enable_lunch_special aktiviert das Lunch-Special, das einen 10% Rabatt auf alle Bestellungen des Tisches gewährt. Der Wert self.lunch_special wird auf True gesetzt, welches anzeigt, dass der Tisch das Lunch-Special aktiviert hat.

Die Methode disable_lunch_special deaktiviert das Lunch-Special wieder, sodass der 10% Rabatt auf alle Bestellungen des Tisches nicht mehr angewendet wird. Der Wert von self.lunch_special wird auf False gesetzt, wodurch das Lunch-Special für den Tisch deaktiviert wird. Eine Bestätigungsmeldung wird ausgegeben, dass das Lunch-Special deaktiviert wurde. Zuerst werden die Gesamtkosten aller Bestellungen auf den Tisch berechnet (alle Order.price für Order objekts in der self.orders Liste wird summiert und in total gespeichert). Desweiteren werden die Savings Anfangs auf 0.0 gesetzt, worauf die Savings jeweils um 10% (0,1) des Gesamtwertes erhöht werden können und diese 10% werden auch von dem Gesamtpreis abgezogen (total), so dass nur 90% vorhanden sind statt die vollen 100%. Insgesamt wieder wollen wir den Gesamtbetrag returnt bekommen, sowie den erstatteten Rabatt auf 2 Nachkommastellen gerrundet.

1.0 for req in self.special_requests if "extra" in req.lower()

Für die Berechnung der Gesamtkosten wird die Methode calculate_total() verwendet, welche den Gesamtbetrag und die Ersparnisse durch gegebene Rabatte berechnen. Daraufhin wird eine Liste erstellt, welche jeweils mit einer fortlaufenden Nummer gekennzeichnet wird. Dafür haben wir enumerate verwendet, welches für jede Bestellung den Index i und das order-Objekt

zurückgibt (Quelle: Built-in Functions — Python 3.13.1 documentation). Es wird eine Liste erzeugt für jede Bestellung.

Die Methode def __str__(self) tritt ein, falls eine Instanz der Klasse als String dargestellt wird. Dadurch gibt sie eine zusammenfassende textuelle Darstellung eines Tisches, seiner Bestellungen und der Gesamtkosten. Hierbei wird auch berücksichtigt, ob Rabatte wie das Lunch-Special aktiv sind, was dann ebenfalls in der Ausgabe angezeigt wird. Timestamp für das Festhalten des Zeitpunktes der Bestellung (%Y..., Quelle: datetime — Basic date and time types — Python 3.13.1 documentation). Woraufhin die Grundstruktur des Ergebnisses erstellt wird. Der Tisch, der Zeitstempel, die Bestellungen und der Gesamtpreis werden im String zusammengefügt, wobei die Zahl „total“ auf zwei Dezimalstellen gerundet wird und mit Zeilenumbrüchen gearbeitet wird. Es muss auch überprüft werden, ob es Rabatte wie z.B. das self.lunch_special aktiviert ist, welches gegebenfalls auch geprintet werden soll. Also der resultierende String, der alle Bestellungen, den Gesamtbetrag und die Ersparnisse enthält (falls das Lunch-Special aktiv ist), wird zurückgegeben in dem Return Befehl.

Also nächstes wurde die Klasse Restaurant codiert, diese tragen zur Verwaltung des Restaurants bei. Zu Beginn wurde wieder def __init__ definiert, dabei ist die Liste aller Tische self.tables als leeres Dictionary initialisiert. Das Menü (self.menu) wird durch den Aufruf der Methode load_menu(menu_file) mit dem Menü gefüllt.

Die Methode def load_menu lädt unser aufgestelltes Menü aus der CSV-Datei heraus und erstellt daraus MenuItem-Objekte. Durch open(menu_file, "r") wird die CSV-Datei im Lese-Modus geöffnet (Quelle: csv — CSV File Reading and Writing — Python 3.13.1 documentation). Wobei beim nächsten Code reader = csv.reader(file, delimiter = ‘;‘) ein CSV-Reader erstellt wird und aus der Datei mit einem ´;´ als Trennzeichen lesen (csv — CSV File Reading and Writing — Python 3.13.1 documentation). Diesen Vorgang haben wir gefühlt für eine korrekte Einlesung der Daten aus der CSV, da diese bei Öffnung mit einem „;“ getrennt sind und damit erkannt wird dass das als Trenner gilt. Daraufhin ist der Sinn der For Schleife, dass diese jede Zeile aus der CSV-Datei abläuft und in der Variable row speichert. Name, _, _, price = row -> das dient dazu, alle Werte aus der CSV-Zeile zu extrahieren, dabei wird der Name und Preis verwendet während andere Werte ignoriert werden („ _“ <- dient dem Zweck, Werte bewusst zu ignorieren). Des Weiteren haben wir uns dazu entschieden, aus der CSV-Datei die Kommata (Preis 12,50 bspw.) durch einen Punkt zu ersetzen und diese in einen float umzuwandeln damit diese korrekt verarbeitet werden (price = float(price.replace(',' , '.'))). Theoretisch wäre es hierbei auch möglich, einfach die genutzte CSV-Datei zu verändern und die Kommata als Punkt zu ersetzen, aber der Allgemeinheit wegen haben wir uns für diese Option entschieden. Daraufhin wurde bei menu.append(MenuItem(name,price)) ein neues MenuItem-Objekt erstellt und der Menu-Liste hinzugefügt. Damit letztendlich bei return eine Liste der MenuItem-Objekte zurückgegeben wird die alle Gerichte erhält.

Die Methode get_table sorgt dafür, dass ein Tisch mit einer bestimmten Nummer zurückgegeben wird. Falls der Tisch noch nicht existiert, wird er neu erstellt und in das Dictionary self.tables eingefügt.

Die Methode def generate_unique_filename(self, table_number) erstellt eine separate Datei für jede Rechnung eines Tisches, sobald der Tisch wiederbelegt wird erhöht sich der Index der Rechnung um 1. base_filename = f"Tisch_{table_number}_Rechnung_" -> Zuweisung von base_filename. Der Index fängt an bei 1 zu zählen. Während der code „while os.path.exists(f"{base_filename}{index}.txt")“ kontrolliert, ob dieser Dateiname existiert wird

dieser falls vorhanden um 1 erhöht und bei return unter der Tischnummer und dem Index gespeichert.

fDie Methode def save_and_pay_bill ermöglicht es, Rechnungen zu speichern und als bezahlt zu markieren. Zuerst wird hierfür der Tisch mit der angegebenen Nummer abgerufen (table = self.get_table(table_number)). Daraufhin werden die Gesamtkosten durch calculate_total() des ausgewählten Tisches berechnet, welches daraufhin die Datei generiert. Die Rechnung wird in der Datei gespeichert (als String Darstellung des Tisches) und wird in der Datei gespeichert (*filename = self.generate_unique_filename(table_number)* UND *with open(filename, "w") as file: file.write(str(table*)). Nach der Ausgabe in der Konsole muss der Tisch und damit die Rechnung noch aus dem System gelöscht werden (del self.tables[table_number]).

Mit der Methode def display_open_bills können alle offenen Rechnungen angezeigt werden, wobei für jeden Tisch die Bestellungen und der Gesamtbetrag aufgelistet werden. Erst wird die Überschrift geprintet („Offene Rechnung“), danach wird für die Rechnung calculate total aufgerufen um den Gesamtpreis zu berechnen (total, _ = table.calculate_total()) woraufhin eine Bestellübersicht für den Tisch ausgegeben wird (order_details = "\n ".join([str(order) for order in table.orders])).

Für die Methode display_menu(self) wird mit enumerate gearbeitet, hier wird das ganze Menü eines Restaurants gezeigt. Die Methode iteriert durch alle Elemente und zeigt sie mit einer fortlaufenden Nummerierung an.

Die nächste Methode def place_order(self, table_number) ermöglicht es, eine Bestellung für einen bestimmten Tisch aufzugeben und lässt dementsprechend so den Benutzer eine Auswahl treffen über seinen Wunsch. Der Code: table = self.get_table(table_number) lässt uns den Tisch abrufen. In der While-Schleife wird erstmal mit self.display_menu() eine Anzeige des Menüs gezeigt, worauf mit choice = self.get_menu_choice() die Auswahl des Benutzers ausgewählt werden und mit if choice.lower() == ´q´ eine Möglichkeit die Bestellung abzubrechen existiert. Order_item = self.create_order_item(choice) dient zur Erstellung einer Bestellung. Durch die if schleife (table.add_order(order_item)) Bestellungen hinzufügen.

Die Methode get_menu_choice(self) ist die Möglichkeit eine Auswahl zu treffen oder den Vorgang zu beenden (q)

Die Methode create_order_item(self, choice) dient dem Zweck, eine Bestellung aufgeben zu können welche in dem Bereich unserer Auswahlmöglichkeiten liegen muss. menu_index = int(choice) - 1 ist für die Umwandlung der Benutzerauswahl in eine Zahl. Um zu überprüfen ob der Index im gültigen Bereich liegt, wurde if 0 <= menu_index < len(self.menu) codiert. Da der Nutzer auch die Möglichkeit auf Sonderwünsche haben kann, wurde special_requests definiert und mit strip gearbeitet, damit die Sonderwünsche formgerecht auftreten (req.strip).

Die Methode manage_table bietet ein interaktives Menü zur Verwaltung eines Tisches, einschließlich der Möglichkeit, Bestellungen hinzuzufügen, zu stornieren, Rabatte anzuwenden, das Lunch-Special zu aktivieren oder zu deaktivieren sowie die Rechnung zu bezahlen und zu speichern. Zusätzlich muss sich der Nutzer für eine der Möglichkeiten entscheiden, da es ansonsten zu einer Fehlermeldung führt.

Die Methode run bildet das Hauptmenü des Systems und ermöglicht es, zwischen der Tischverwaltung, der Anzeige offener Rechnungen und dem Beenden des Programms zu wählen. Benutzeraktionen werden überprüft und entsprechend verarbeitet, um eine fehlerfreie Bedienung zu gewährleisten.

Dokumentation – Tests

Die Datei testcases.py enthält Testfälle für die verschiedenen Funktionen und Klassen des Restaurant-Management-Systems, um deren Korrektheit und Verhalten zu überprüfen:

test_table_init

Testet die Initialisierung der Table-Klasse. Dabei wird überprüft, ob die Attribute table_number, orders, discount und lunch_special korrekt gesetzt werden.

- **Eingabe**: Table-Objekte mit unterschiedlichen Tischnummern.
- **Erwartetes Verhalten**: Die Tischnummer wird korrekt zugewiesen, die Liste der Bestellungen ist leer, der Rabatt beträgt 0 %, und lunch_special ist False.
- **Ausgabe**: Konsolenausgabe der Attribute der erstellten Tische.

test_add_order

Die Funktion testet die add_order-Methode der Table-Klasse.

- **Eingabe**: Hinzufügen von OrderItem-Objekten zu einem Tisch.
- **Erwartetes Verhalten**: Die Bestellungen werden korrekt zur Liste hinzugefügt.
- **Ausgabe**: Konsolenausgabe der Bestellungen nach jedem Hinzufügen.

test_restaurant_init

Dieser Test überprüft die Initialisierung der Restaurant-Klasse, insbesondere das Laden des Menüs aus einer CSV-Datei.

- **Eingabe**: Pfad zu einer Menüdatei (z. B. "food.csv").
- **Erwartetes Verhalten**: Das Menü wird korrekt aus der Datei geladen und in Form von MenuItem-Objekten gespeichert.
- **Ausgabe**: Konsolenausgabe des geladenen Menüs.

test_load_menu

Die load_menu-Methode wird getestet, um sicherzustellen, dass sie CSV-Dateien korrekt liest und MenuItem-Objekte erstellt.

- **Eingabe**: Verschiedene CSV-Dateien.
- **Erwartetes Verhalten**: Die Dateien werden korrekt geparst und die entsprechenden Objekte erstellt.
- **Ausgabe**: Konsolenausgabe der geladenen Menüelemente.

test_get_table

Hier wird die Methode get_table der Restaurant-Klasse getestet, die Tische abruft oder erstellt.

- **Eingabe**: Tischnummern.
- **Erwartetes Verhalten**: Ein neuer Tisch wird erstellt, wenn er noch nicht existiert, oder der vorhandene Tisch wird zurückgegeben.

- **Ausgabe**: Konsolenausgabe der zurückgegebenen Tische.

test_generate_unique_filename

Diese Funktion testet die generate_unique_filename-Methode der Restaurant-Klasse.

- **Eingabe**: Tischnummern.
- **Erwartetes Verhalten**: Ein eindeutiger Dateiname wird für jede Tischnummer generiert.
- **Ausgabe**: Konsolenausgabe der generierten Dateinamen.

test_save_and_pay_bill

Der Test überprüft das Speichern und Bezahlen von Rechnungen.

- **Eingabe**: Bestellungen für Tische und Tischnummern.
- **Erwartetes Verhalten**: Die Rechnungen werden gespeichert, und die Tische werden geleert.
- **Ausgabe**: Konsolenausgabe der gespeicherten Rechnungen und des Zahlbetrags.

test_display_open_bills

Testet die Methode zum Anzeigen offener Rechnungen.

- **Eingabe**: Bestellungen für mehrere Tische.
- **Erwartetes Verhalten**: Die offenen Rechnungen aller Tische werden korrekt angezeigt.
- **Ausgabe**: Konsolenausgabe der offenen Rechnungen.

test_display_menu

Die Funktion testet die Anzeige des Menüs.

- **Eingabe**: Kein spezifischer Input.
- **Erwartetes Verhalten**: Das Menü wird vollständig und korrekt angezeigt.
- **Ausgabe**: Konsolenausgabe des Menüs.

test_place_order

Testet die Methode zum Aufgeben von Bestellungen.

- **Eingabe**: Tischnummern und Menüauswahl.
- **Erwartetes Verhalten**: Die ausgewählten Bestellungen werden korrekt hinzugefügt.
- **Ausgabe**: Konsolenausgabe der hinzugefügten Bestellungen.

test_manage_table

Hier wird die Verwaltung eines Tisches getestet.

- **Eingabe**: Tischnummern und Benutzeraktionen (z. B. Hinzufügen von Bestellungen, Anwenden von Rabatten).
- **Erwartetes Verhalten**: Die entsprechenden Aktionen werden korrekt ausgeführt.

- **Ausgabe**: Konsolenausgabe der Änderungen.

test_run

Diese Funktion testet die Hauptschnittstelle des Systems.

- **Eingabe**: Benutzeraktionen im Hauptmenü.
- **Erwartetes Verhalten**: Das System reagiert korrekt auf die Eingaben.
- **Ausgabe**: Konsolenausgabe der Benutzeraktionen.

test_get_menu_choice

Testet die Benutzereingabe für Menüauswahlen.

- **Eingabe**: Benutzereingaben.
- **Erwartetes Verhalten**: Die Eingaben werden korrekt verarbeitet.
- **Ausgabe**: Konsolenausgabe der Benutzerwahl.

test_create_order_item

Die Methode zum Erstellen von Bestellungen wird getestet.

- **Eingabe**: Menüauswahl und Sonderwünsche.
- **Erwartetes Verhalten**: Die Bestellungen werden korrekt erstellt.
- **Ausgabe**: Konsolenausgabe der erstellten Bestellungen.

test_cancel_order

Testet das Stornieren von Bestellungen.

- **Eingabe**: Index der zu stornierenden Bestellung.
- **Erwartetes Verhalten**: Die Bestellung wird korrekt aus der Liste entfernt.
- **Ausgabe**: Konsolenausgabe des Erfolgs oder Misserfolgs.

test_apply_discount

Testet das Anwenden von Rabatten.

- **Eingabe**: Rabattprozentsätze.
- **Erwartetes Verhalten**: Der Rabatt wird korrekt angewendet.
- **Ausgabe**: Konsolenausgabe der Rabattwerte.

test_enable_lunch_special / test_disable_lunch_special

Diese Funktionen testen das Aktivieren und Deaktivieren des Lunch-Specials.

- **Eingabe**: Keine spezifische Eingabe.
- **Erwartetes Verhalten**: Der Status des Lunch-Specials wird korrekt gesetzt oder zurückgesetzt.
- **Ausgabe**: Konsolenausgabe des aktuellen Status.

test_calculate_total

Testet die Berechnung der Gesamtkosten eines Tisches.

- **Eingabe**: Bestellungen und Rabattwerte.
- **Erwartetes Verhalten**: Die Gesamtkosten und Einsparungen werden korrekt berechnet.
- **Ausgabe**: Konsolenausgabe der Gesamtkosten und Rabatte.

test_table_str

Testet die String-Repräsentation eines Tisches.

- **Eingabe**: Bestellungen und Rabatte.
- **Erwartetes Verhalten**: Die Repräsentation enthält alle relevanten Informationen.
- **Ausgabe**: Konsolenausgabe der String-Repräsentation.